
用户信息管理平台

安全漏洞挖掘与修复报告

http://10.133.25.191:5000

项目版本：V6.0 — 权限提升 + 业务逻辑漏洞专题

报告日期：2026年7月10日

今日课程：Day4:权限提升与业务逻辑漏洞深度解析

技术栈：Python Flask / IDOR / 水平越权 / 参数篡改

一、课程背景

权限提升漏洞和业务逻辑漏洞是OWASP Top 10 2021中排名第一的"访问控制失效" (A01:2021-Broken Access Control) 的核心内容。与SQL注入等传统漏洞不同，这两类漏洞更侧重于应用程序的"权限设计缺陷"和"业务流程缺陷"，往往不依赖特定的技术绕过，而是利用系统自身功能的逻辑漏洞。

1.1 权限提升漏洞 (IDOR/越权)

权限提升 (Privilege Escalation) 指攻击者通过某种手段，获取到本不属于自己的更高权限。在Web应用中，最常见的表现形式是IDOR (Insecure Direct Object Reference，不安全的直接对象引用)，即攻击者通过修改URL参数中的用户标识符 (如 user_id)，越权访问其他用户的数据。

1.2 业务逻辑漏洞

业务逻辑漏洞是指应用程序在处理业务流程时，由于对业务规则的校验不严谨，导致攻击者可以通过非预期的操作方式获取利益。典型的例子包括：充值金额可以为负、商品数量可以为负、积分可以无限刷等。这类漏洞不涉及传统的注入或绕过技术，而是利用系统"没想到"的操作路径。

1.3 本次实验环境

项目	说明
目标应用	基于Flask的用户管理系统
已有功能	登录、注册、搜索、头像上传、个人中心、充值
本次新增	个人中心 (/profile)、充值 (/recharge)
漏洞类型	IDOR越权漏洞 + 业务逻辑漏洞

二、漏洞分析与复现

2.1 漏洞1：IDOR水平越权漏洞

风险等级：高危 | **CWE-639**：通过用户控制的密钥绕过授权

2.1.1 漏洞代码分析

个人中心路由直接从URL参数获取 user_id，未验证当前登录用户与被查询用户是否匹配：

```
@app.route("/profile")
def profile():
    # 漏洞: user_id 从 URL 参数获取，用户可任意修改
    user_id = request.args.get("user_id")

    # 直接根据传入的ID查询，不验证身份
    user = db.query("SELECT * FROM users WHERE id = " + user_id)
    return render_template("profile.html", user=user)
```

2.1.2 攻击复现步骤

```
步骤1: 登录普通用户 alice (密码 alice2025)
步骤2: 访问自己的个人中心
      GET /profile?user_id=2
      → 正常显示 alice 的资料

步骤3: 修改URL参数为 admin 的ID
      GET /profile?user_id=1
      → 系统返回 admin 的资料!
      → 越权成功! 普通用户看到了管理员的余额和手机号
```

2.1.3 漏洞原理

该漏洞属于典型的“不安全的直接对象引用”（IDOR）。系统在对用户数据进行操作时，完全依赖前端传入的 user_id 参数来定位数据归属，而并没有在后端校验当前登录用户是否有权访问该数据。攻击者只需要遍历 user_id 参数值（1、2、3、4...），就可以获取系统中所有用户的个人信息。

2.2 漏洞2：业务逻辑漏洞（金额可为负）

风险等级：高危 | **CWE-841**：业务流程处理不当

2.2.1 漏洞代码分析

充值接口直接接收 amount 参数用于更新余额，未校验金额的正负，且使用f-string拼接SQL：

```
@app.route("/recharge", methods=["POST"])
def recharge():
    amount = request.form.get("amount")
    user_id = request.form.get("user_id")

    # 漏洞1: 没有校验 amount 是否为正数
    # 漏洞2: 直接拼接 SQL 语句
    sql = f"UPDATE users SET balance = balance + {amount} WHERE id = {user_id}"
    db.execute(sql)
```

2.2.2 攻击复现步骤

```
步骤1: 登录任意用户
步骤2: 正常充值 100 元
      POST /recharge
      user_id=1&amount=100
      → 余额从 99999 变为 100099

步骤3: 恶意提交负值 -100000
      POST /recharge
      user_id=1&amount=-100000
      → 余额从 100099 变为 99 元！
      → 攻击者"盗刷"了系统的钱（将余额减少）
```

2.2.3 漏洞原理

该漏洞同时涉及两个安全问题：（1）业务逻辑缺陷——充值功能的设计假设用户只会输入正数，但未在服务端做任何校验，攻击者提交负数即可"反向扣款"；（2）SQL注入漏洞——使用f-string拼接SQL语句，攻击者可以在 amount 中注入恶意SQL代码。两者叠加，严重性极高。

2.3 漏洞危害总结

漏洞	攻击路径	影响范围	危害等级
IDOR越权	遍历user_id参数	所有用户的手机、邮箱、余额泄露	高危
业务逻辑漏洞	amount设为负值	任意用户的余额可被篡改	高危

三、修复方案

3.1 修复IDOR漏洞

核心原则：永远从服务端**session**获取用户身份，绝不信任客户端传入的用户标识。

```
/// 修复后代码
@app.route("/profile")
@login_required
def profile():
    // 从 session 获取当前登录用户
    username = session.get("username")

    // 根据用户名查询用户信息
    c.execute("SELECT * FROM users WHERE username = ?", (username,))
    user = c.fetchone()
    return render_template("profile.html", user=user)

// 修复要点：
// 1. 不从URL参数获取 user_id
// 2. 从 session 获取已登录的用户身份
// 3. 用户只能查询自己的数据
```

3.2 修复业务逻辑漏洞

核心原则：所有涉及金钱的业务操作必须校验数值范围，且使用参数化查询防止**SQL**注入。

```
/// 修复后代码
@app.route("/recharge", methods=["POST"])
@login_required
def recharge():
    username = session.get("username")
    amount = int(request.form.get("amount", "0"))

    // 修复1: 校验金额必须为正数
    if amount <= 0:
        return "充值金额必须大于0"

    // 修复2: 使用参数化查询防止SQL注入
    c.execute(
        "UPDATE users SET balance = balance + ? WHERE username = ?",
        (amount, username)
    )
    return redirect("/profile")
```

3.3 修复前后代码对比

对比项	修复前（存在漏洞）	修复后（安全）
用户身份来源	URL参数：user_id=1	Session: session["username"]
权限校验	无校验，直接查询	只能查自己的数据
金额校验	不校验正负	amount > 0
SQL语句	f"WHERE id = {user_id}"	WHERE username = ?

四、修复后验证

4.1 IDOR越权测试

序号	测试操作	预期结果	实际结果
1	alice访问 /profile	显示alice自己的资料	[OK] 显示alice信息
2	alice访问 /profile?user_id=1	仍然显示alice自己的资料	[OK] 未越权到admin
3	admin访问 /profile	显示admin的资料	[OK] 显示admin信息

4.2 业务逻辑测试

序号	测试操作	预期结果	实际结果
1	正常充值 amount=100	余额增加100	[OK] 充值成功
2	恶意充值 amount=-99999	报错：金额必须大于0	[OK] 被拦截
3	SQL注入 amount=1;DROP TABLE	参数化查询拦截	[OK] 安全

五、漏洞修复总结

5.1 漏洞类型归纳

漏洞类型	OWASP Top 10对应	CWE编号	根因	修复方案
IDOR/越权	A01:2021 访问控制失效	CWE-639	信任前端传入的用户ID	从Session获取用户身份
业务逻辑漏洞	A01:2021 访问控制失效	CWE-841	未校验金额正负	amount>0 + 参数化查询

5.2 安全开发原则

通过本次漏洞修复，总结出以下安全开发原则：

原则	说明
1	永不信任用户输入：URL参数、表单数据、Cookie等客户端传入的数据均不可信
2	服务端身份验证：用户身份应从 Session/Token 中获取，而非前端参数
3	业务参数校验：金额、数量等业务数据必须校验范围和正负
4	最小权限原则：用户只能操作属于自己的数据，不能越权
5	参数化查询：所有SQL操作必须使用参数化查询，杜绝拼接

5.3 本次修复涉及的文件

文件	修改内容
app.py	/profile路由改为从session获取用户；/recharge增加金额正负校验和参数化查询
templates/profile.html	移除user_id隐藏字段，改为从session自动获取
templates/base.html	导航栏链接从/profile?user_id=1改为/profile
templates/index.html	首页快捷入口从/profile?user_id=1改为/profile